1	2	3	4	$\sum$

Freie Universität Berlin

Algorithmen, Datenstrukturen und Datenabstraktion (WiSe 18/19)

Docent: Prof. Dr. Wolfgang Mulzer

Tutorin: Valeria Zahoransky; Tutorial: 07

Exercise 07

Release: 26.11.2018; Due: 07.12.2018

Viktoriya Kraleva, Samuel Enderwitz

---

1 Hashings: Implementation (__/10)

(a) *Read the documentation of* `hashCode()` *in* `java.lang.Object`*. Which properties need to be satisfied? Why should you overwrite the method in your own classes?*

1) Wenn die Funktion `hashCode()` mehrmals auf das selbe Objekt während der Laufzeit ausgeführt wird, wird jedes mal die gleiche Zahl zurückgegeben. Wenn aber dazwischen Änderungen am Objekt vorgenommen werden, die auch die `equals(Object)` Funktion zum unterscheiden verwendet wird, ändert sich auch der HashCode.
Der Rückgabewert kann sich aber von Ausführung zu Ausführung des selben Programms unterscheiden.

2) Das Ergebnis von `equals(Object)` und der Vergleich der beiden HashCodes der Objekte sind immer gleich.

Man schreib eine Klasse und erzeuge zwei Objekte daraus. Nun kann es sein, dass die Objekte teilweise unterscheidliche Werte haben, wir aber dennoch sagen wollen, dass diese zwei Objekte gleich sind, da die wichtigen, also unterscheidbaren Werte unterscheidlich sind. Dabei soll dann der selber HashCode raus kommen.

(b) *Implement hashing with chaining, as described in class. Use* `hashCode()`*. For the linked lists, you may use the Java-library. The hash-table should grow dynamically as soon as a certain load factor is reached. Perform experiments with your implementation. How do the access times depend on the load factor? Compare two different hash functions (these may be hard-coded for a fixed table size).*

Anmerkung: Aufgabe 4(c) wurde direkt hier mit Implementiert und kann über Parameter im Konstruktor ausgewählt werden.

```java
package de.smuu.add.e07;

public class NodeElement<K, V> {

    private final K key;
    private V value;

```

```java
    public NodeElement(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public Object getKey() {
        return key;
    }

    public Object getValue() {
        return value;
    }

    public void setValue(V value) {
        this.value = value;
    }

    @Override
    public String toString() {
        return "NodeElement{key=" + key + ",value=" + value + "}";
    }
}
```

Listing 1: NodeElement

```java
package de.smuu.add.e07;

import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.math.BigInteger;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.Arrays;
import java.util.LinkedList;

public class HashingWithChaining<K, V> implements HashInterface {

    private LinkedList<V>[] hashTable;
    private double maxLoadFactor;
    private int prim;
    private String crypt;

    public HashingWithChaining() {
        this(1, "");
    }

    public HashingWithChaining(String crypt) {
        this(1, crypt);
    }

    public HashingWithChaining(double maxLoadFactor) {
        this(maxLoadFactor, 1);
    }

    public HashingWithChaining(double maxLoadFactor, String crypt) {
        this(maxLoadFactor, 1, crypt);
```

```java
    }

    public HashingWithChaining(double maxLoadFactor, int size) {
        this(maxLoadFactor, size, 0);
    }

    public HashingWithChaining(double maxLoadFactor, int size, String crypt) {
        this(maxLoadFactor, size, 0, crypt);
    }

    /**
     * @param maxLoadFactor
     * @param size
     * @param prim          0 - deactivate; >0 must be prim
     */
    public HashingWithChaining(double maxLoadFactor, int size, int prim) {
        this(maxLoadFactor, size, prim, "");
    }

    public HashingWithChaining(double maxLoadFactor, int size, int prim, String crypt) {
        if (size <= 0) {
            throw new IllegalArgumentException("getSize must be positiv");
        }
        if (prim != 0) {
            if (prim < size) {
                throw new IllegalArgumentException("prim must be higher than the getSize");
            }
            if (!isPrim(prim)) {
                throw new IllegalArgumentException("prim is not a prim number");
            }
        }
        this.prim = prim;
        this.maxLoadFactor = maxLoadFactor;
        hashTable = new LinkedList[size];
        for (int i = 0; i < size; i++) {
            hashTable[i] = new LinkedList<>();
        }
        this.crypt = crypt;
    }

    @Override
    public void add(Object key, Object value) {
        if (getLoadFactor() >= maxLoadFactor) {
            resize();
        }
        int hash = getHash(key);
        LinkedList linkedList = hashTable[hash];
        for (Object element : linkedList) {
            NodeElement nodeElement = (NodeElement) element;
            if (nodeElement.getKey().equals(key)) {
                nodeElement.setValue(value);
```

```
                return;
            }
        }
        linkedList.add(new NodeElement<>(key, value));
    }

    @Override
    public Object get(Object key) {
        int hash = getHash(key);
        LinkedList<V> linkedList = hashTable[hash];
        for (Object object : linkedList) {
            NodeElement nodeElement = (NodeElement) object;
            if (nodeElement.getKey() == key) {
                return nodeElement.getValue();
            }
        }
        throw new ElementNotFoundException();
    }

    @Override
    public Object remove(Object key) {
        int hash = getHash(key);
        LinkedList<V> linkedList = hashTable[hash];
        for (Object object : linkedList) {
            NodeElement nodeElement = (NodeElement) object;
            if (nodeElement.getKey() == key) {
                linkedList.remove(object);
                return nodeElement.getValue();
            }
        }
        throw new ElementNotFoundException();
    }

    @Override
    public int getSize() {
        int size = 0;
        for (LinkedList<V> list : hashTable) {
            size += list.size();
        }
        return size;
    }

    @Override
    public boolean isEmpty() {
        return getSize() == 0;
    }

    private int getHash(Object key) {
        return getHash(key, hashTable.length);
    }

    private int getHash(Object key, int mod){
        if (!crypt.equals("")) {
            MessageDigest md = null;
            try {
                md = MessageDigest.getInstance(crypt);
            } catch (NoSuchAlgorithmException e) {
```

```
                e.printStackTrace();
            }
            assert md != null;
            md.update(toByteArray(key));
            int pos = new Integer(String.valueOf(new BigInteger(md.digest()).mod(new BigInteger(String.valueOf(mod)))));
            return pos;
        } else {
            if (prim == 0) {
                return key.hashCode() % mod;
            }
            return (key.hashCode() % prim) % mod;
        }
    }

    private byte[] toByteArray(Object obj) {
        try {
            ByteArrayOutputStream bos = new ByteArrayOutputStream();
            ObjectOutputStream oos = new ObjectOutputStream(bos);
            oos.writeObject(obj);
            //oos.flush();
            byte[] data = bos.toByteArray();
            bos.close();
            oos.close();
            return data;
        } catch (IOException e) {
            e.printStackTrace();
        }
        return null;
    }

    private void resize() {
        LinkedList<V>[] hashTable = new LinkedList[this.hashTable.length * 2];
        for (int i = 0; i < hashTable.length; i++) {
            hashTable[i] = new LinkedList<>();
        }
        prim = getPrimHigherThan(this.hashTable.length * 2);

        for (LinkedList<V> list : this.hashTable) {
            for (Object element : list) {
                NodeElement nodeElement = (NodeElement) element;
                int hash = getHash(nodeElement.getKey(), hashTable.length);
                LinkedList linkedList = hashTable[hash];
                linkedList.add(element);
            }
        }
        this.hashTable = hashTable;
    }

    public double getLoadFactor() {
        return ((double) getSize()) / ((double) hashTable.length);
    }

    private boolean isPrim(int i) {
        if (i <= 0) {
```

```java
            throw new IllegalArgumentException("primes not negative");
        }
        if (i == 1) {
            return false;
        }
        if (i == 2) {
            return true;
        }
        if (i % 2 == 0) {
            return false;
        }
        for (int j = 3; j < i / 2; j += 2) {
            if (i % j == 0) {
                return false;
            }
        }
        return true;
    }

    private int getPrimHigherThan(int i) {
        while (true) {
            i++;
            if (isPrim(i)) {
                return i;
            }
        }
    }

    public int getCollisions() {
        int coll = 0;
        for (LinkedList<V> list : hashTable) {
            if (list.size() > 1) {
                coll += list.size() - 1;
            }
        }
        return coll;
    }

    @Override
    public String toString() {
        return Arrays.toString(hashTable);
    }
}
```

Listing 2: HashingWithChaining

```java
package de.smuu.add.e07;

public class HashingMain {

    public static void main(String[] args) {

        int amount = 15;

        int coll_h1 = 0;
        int coll_h2 = 0;
        int coll_h3 = 0;
```

```java
        int coll_h4 = 0;

        for (int i = 0; i < amount; i++) {

            System.out.println((i+1) + ". Durchlauf");
            System.out.println();

            // Hashfunktion h(x) = x mod N
            HashingWithChaining<Integer, String> h1 = new HashingWithChaining<>(0.75, 10);
            // Hashfunktion h(x) = (x mod p) mod N
            HashingWithChaining<Integer, String> h2 = new HashingWithChaining<>(0.75, 10, 7919);
            // Kryptografische Hashfunktion MD5
            HashingWithChaining<Integer, String> h3 = new HashingWithChaining<>(0.75, "MD5");
            // Kryptografische Hashfunktion SHA-512
            HashingWithChaining<Integer, String> h4 = new HashingWithChaining<>(0.75, "SHA-512");

            for (int j = 0; j < 1000; j++) {
                int rand = (int) (Math.random() * 10000);
                h1.add(rand, j);
                h2.add(rand, j);
                h3.add(rand, j);
                h4.add(rand, j);
            }

            int coll_h1_this = h1.getCollisions();
            int coll_h2_this = h2.getCollisions();
            int coll_h3_this = h3.getCollisions();
            int coll_h4_this = h4.getCollisions();
            coll_h1 += coll_h1_this;
            coll_h2 += coll_h2_this;
            coll_h3 += coll_h3_this;
            coll_h4 += coll_h4_this;

            System.out.println("Hashtabels mit " + h1.getSize() + " Elementen");
            System.out.println();
            System.out.println("LadeFaktor h1: " + h1.getLoadFactor());
            System.out.println("LadeFaktor h2: " + h2.getLoadFactor());
            System.out.println("LadeFaktor h3: " + h3.getLoadFactor());
            System.out.println("LadeFaktor h4: " + h4.getLoadFactor());
            System.out.println();
            System.out.println("Kollisionen h1: " + coll_h1_this);
            System.out.println("Kollisionen h2: " + coll_h2_this);
            System.out.println("Kollisionen h3: " + coll_h3_this);
            System.out.println("Kollisionen h4: " + coll_h4_this);
            System.out.println();
        }

        System.out.println("Durchschnitt Kollisionen h1: " + coll_h1 / amount);
        System.out.println("Durchschnitt Kollisionen h2: " + coll_h2 / amount);
        System.out.println("Durchschnitt Kollisionen h3: " + coll_h3 /
```

```
    amount);
        System.out.println("Durchschnitt Kollisionen h4: " + coll_h4 /
    amount);



     }
}
```

Listing 3: HashingMain

Wenn wir einen hohen Belegungsfaktor haben, dann ist die Wahrscheilichkeit größer, dass wir eine Verkettung an den verschiedenen Stellen haben. Dadurch kann es bei einem Zugriff auf ein Element zu einer langen Zugriffszeit kommen, da ggf. die gesamte verkettete Liste durchsucht werden muss.

Wir haben eine Test mit 4 verschiedenen Hashfunktionen durchgeführt.

1) Hashfunktion 1:
   Einfache Hashfunktion $k \mod N$
2) Hashfunktion 2:
   Hashfunktion mit Primzahl $(k \mod p) \mod N$
3) Hashfunktion 3:
   Kryptografische Hashfunktion MD5
4) Hashfunktion 4:
   Kryptografische Hashfunktion SHA-512

Den Test haben wir so geschrieben, dass wir Zufallszahlen als Schlüssel in die Hashfunktionen geben und am Ende die Anzahl der Kollisionen zählen. Um den Test etwas repäsentativer zu machen, haben wir den Test mehrmals durchgeführt und uns dann den Durchschnitt berechnen lassen.

```
Durchschnitt Kollisionen h1: 242
Durchschnitt Kollisionen h2: 242
Durchschnitt Kollisionen h3: 194
Durchschnitt Kollisionen h4: 193
```

Listing 4: Durchschnittliche Kollisionen

Hier sieht man schnell, dass es für Kollisionen keinen Unterschied macht, ob man eine Hashfunktion mit oder Primzahl schreibt. Zwischen einer normalen Hashfunktion und einer kryptografischen Hashfunktion sieht man einen großen Unterschied. Die beiden kryptografischen Hashfunktionen unterscheiden sich kaum. Der wesentliche Unterschied zwischen den beiden ist die größe des Hashwertes.

2 Analysis of Skip Lists (__/10)

Let L be a skip list with n entries. Show the following:

(a) *The expected number of nodes in L is $O(n)$.*

Die Wahrscheinlichkeit, dass ein Element in der Liste L_i vorhanden ist, ist $\frac{1}{2^i}$. Die Begrüngung dazu ist dass die Anzahl der Elemente auf der Ebene L_{i+1} die Hälfte der Elemente auf Ebene L_i ist.

Die Anzahl der Listen in einer Skip List ist laut Vorlesung $\log n$

Wir addieren die Anzahl der Elemente auf den einzelnen Listen miteinander auf:

$$\sum_{i=1}^{\log_2 n} \frac{n}{2^i} = n \cdot (1 + \frac{1}{2} + \ldots + \frac{1}{2^{\log_2 n}}) \leq 2n$$

Somit haben wir eine obere Schranke der Knoten in einer Skip-Liste. Somit gilt $2n \in O(n)$ (Hier ohne Beweis).

(b) *With probability at most $n/2^{j-1}$, the list L has at least j horizontal lists.* Hint*: For events $A_1, \ldots, A_\ell$, we have* $\Pr[A_1 \cup \cdots \cup A_\ell] \leq \sum_{i=1}^{\ell} \Pr[A_i]$ *(this is called the* union bound).

$Pr[\text{Knoten m ist auf Ebene j}] = \frac{1}{2^{j-1}}$

Damit eine Skipliste mindestens j Ebenen hat, muss auf Ebene j mindestens ein Element sein. ($j-1$, da die erste Ebene mit 0 betitelt wird)

Insgesamt haben wir n Einträge in der Skipliste.

$$\begin{aligned}
Pr[\text{n Elemente auf Ebene 1}] &= Pr[A_1] = \frac{1}{2^{1-1}} = 1 \\
Pr[\text{n Elemente auf Ebene 2}] &= Pr[A_2] = \frac{1}{2^{2-1}} = \frac{1}{2} \\
Pr[\text{n Elemente auf Ebene 3}] &= Pr[A_3] = \frac{1}{2^{3-1}} = \frac{1}{4} \\
Pr[\text{n Elemente auf Ebene l}] &= Pr[A_l] = \frac{1}{2^{l-1}} \\
Pr[\text{n Elemente auf Ebene l+1}] &= Pr[A_l] = \frac{1}{2^{l}}
\end{aligned}$$

Wir wissen, dass $Pr[A_l] > Pr[A_{l+1}]$ und $Pr[A_{l+1}] > Pr[A_{l+2}]$.

Im nächste Schritt betrachten wir die Wahrscheinlichkeit für alle j Ebenen. Also Verenigen wir alle Wahrscheinlichkeiten:

$$\begin{aligned}
&Pr[A_1 \cup A_2 \cup \ldots \cup A_l] \\
&= Pr(A_1) + Pr(A_2) - Pr(A_1 \cap A_2) + Pr(A_1) + Pr(A_3) + \\
&+ Pr(A_1 \cap A_3) + Pr(A_1 \cap A_3 \cap A_2) \ldots
\end{aligned}$$

Durch die Abschätzung im Hinweis können wir folgenderweiße abschätzen:

$$Pr[A_1 \cup A_2 \cup \ldots \cup A_l] \leq \sum_{i=l}^{\log_2 n} Pr[A_i] \leq \sum_{i=l}^{n} Pr[A_i]$$
$$\leq \frac{n}{2^{j-1}}$$

Somit haben wir bewiesen, dass $\frac{n}{2^{j-1}}$ die obere Schranke für die Wahrscheinlichkeit ist, dass eine Skipliste j Ebenen hat.

Die Anzahl der Ebenen ist im Mittel $\log_2 n$. Es besteht aber auch die Wahrscheinlichkeit, dass es mehr oder weniger Ebenen sind. Dabei ist die obere Schrake n.

3 Implementation of an Ordered Dictionary (__/10)

(a) *Write a Java-interface for the abstract data type* `OrderedDictionary` *from class. Provide comments to specify the operations. Implement the interface using a skip list.* Hint: *You can find pseudocde for the skip list operations on the website.*

```java
package de.smuu.add.e07;

public interface OrderedDictionary<K, V> {

    /**
     * returns List contains all predecessors nodes of key k
     * @param key lowest key
     * @return List of al predecessor nodes of key k
     */
    Stack<SkipListElement> search(K key);

    /**
     * puts an element to dictionary
     * @param key key of element
     * @param value value of element
     */
    void put(K key, V value);

    /**
     * returns the value of a specific key
     * @param key key of wanted element
     * @return value of a specific key
     */
    V get(K key) throws ElementNotFoundException;

    /**
     * removes and return the value of a specific key
     * @param key key of wanted element
     * @return value of a specific key
     * @throws ElementNotFoundException
     */
    V remove(K key) throws ElementNotFoundException;

    /**
     * get the value of the next higher key
```

```
     * @param key lowest key
     * @return value of the next higher key
     * @throws NoElementInRangeException
     */
    V getPred(K key) throws NoElementInRangeException;

    /**
     * get the value of the next lower key
     * @param key highest key
     * @return value of the next lower key
     * @throws NoElementInRangeException
     */
    V getSucc(K key) throws NoElementInRangeException;

    /**
     * @return amount of elements in the dictionary
     */
    int getSize();

    /**
     * @return if dictionary is empty
     */
    boolean isEmpty();
}
```

Listing 5: OrderedDictionary

```
package de.smuu.add.e07;

public class NoElementInRangeException extends RuntimeException{
}
```

Listing 6: NoElementInRangeException

```
package de.smuu.add.e07;

public class SkipListElement<K extends Comparable<K>, V> {

    //key and value = null => dummy element at begin
    //value = null => not element on higher level

    private final K key;
    private V value;

    private SkipListElement pred;
    private SkipListElement succ;
    private SkipListElement down;

    public SkipListElement(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() {
        return key;
    }

```

```java
    public Object getValue() {
        return value;
    }

    public void setValue(V value) {
        this.value = value;
    }

    public SkipListElement getPred() {
        return pred;
    }

    public void setPred(SkipListElement pred) {
        this.pred = pred;
    }

    public SkipListElement getSucc() {
        return succ;
    }

    public void setSucc(SkipListElement succ) {
        this.succ = succ;
    }

    public SkipListElement getDown() {
        return down;
    }

    public void setDown(SkipListElement down) {
        this.down = down;
    }

    public boolean isDummy() {
        return key == null && value == null;
    }

    public boolean isHigher() {
        return value == null;
    }

    @Override
    public String toString() {
        return "NodeElement{key=" + key + ",value=" + value + ",down=" +
    (down!=null) + "}";
    }

    public boolean isLowerThan(K key) {
        if (isDummy()) {
            return true;
        }
        return this.key.compareTo(key) < 0;
    }

    public boolean isHigherThan(K key) {
        if (isDummy()) {
            return true;
        }
```

```java
        return this.key.compareTo(key) > 0;
    }

    public boolean isEqualTo(K key) {
        if (isDummy()) {
            return false;
        }
        return this.key.compareTo(key) == 0;
    }
}
```

Listing 7: SkipListElement

```java
package de.smuu.add.e07;

import java.util.Random;

public class SkipList<K extends Comparable<K>, V> implements OrderedDictionary<K, V> {

    private SkipListElement firstHighest;
    private SkipListElement firstLowest;
    private SkipListElement lastLowest;

    public SkipList() {
        firstHighest = new SkipListElement(null, null);
        firstLowest = firstHighest;
        lastLowest = new SkipListElement(null, null);
        firstLowest.setSucc(lastLowest);
        lastLowest.setPred(firstLowest);
    }

    @Override
    public Stack<SkipListElement> search(K key) {
        Stack<SkipListElement> stack = new DynamicArrayStack<>();
        SkipListElement element = firstHighest;

        if (firstHighest.getSucc() == null) {
            stack.push(element);
            return stack;
        }
        do {
            while (element.getSucc() != null && (element.getSucc().isLowerThan(key) || element.getSucc().isEqualTo(key))) {
                element = element.getSucc();
            }
            stack.push(element);
            element = element.getDown();
        } while (element != null);

        return stack;
    }

    @Override
    public void put(K key, V value) {
        Stack<SkipListElement> stack = search(key);
        SkipListElement element = stack.pop();
```

```java
        if (!element.isDummy()) {
            if (element.isEqualTo(key)) {
                element.setValue(value);
                return;
            }
        }
        //Push back for using
        stack.push(element);

        int level = 0;
        boolean coin;

        do {
            level++;
            Random random = new Random();
            coin = random.nextBoolean();
        } while (coin);

        SkipListElement newElement = new SkipListElement(key, value);
        SkipListElement newNode = newElement;

        for (int i = 0; i < level; i++) {
            if (!stack.isEmpty()) {
                if (newNode.isHigher()) {
                    newNode.setDown(newElement);
                    newElement = newNode;
                }
                element = stack.pop();
                newNode.setSucc(element.getSucc());
                newNode.setPred(element);
            } else {
                //new ebene
                SkipListElement newBegin = new SkipListElement(null, null);
                newBegin.setDown(firstHighest);
                firstHighest = newBegin;
                element = new SkipListElement(key, null);
                firstHighest.setSucc(newNode);
                newNode.setDown(newElement);
                newElement = newNode;
                newNode.setPred(firstHighest);
            }
            element.setSucc(newNode);
            newNode = new SkipListElement(key, null);
        }
    }

    @Override
    public V get(K key) throws ElementNotFoundException {
        Stack<SkipListElement> stack = search(key);
        SkipListElement element = stack.pop();
        if (element.isEqualTo(key)) {
            return (V) element.getValue();
        }
        throw new ElementNotFoundException();
    }

```

```java
    @Override
    public V remove(K key) throws ElementNotFoundException {
        Stack<SkipListElement> stack = search(key);
        SkipListElement element = stack.pop();
        SkipListElement tmp = element;
        if (!element.isEqualTo(key)) {
            throw new ElementNotFoundException();
        }
        //push back for later use
        stack.push(element);

        while (!stack.isEmpty()) {
            element = stack.pop();
            if (!element.isDummy() && element.isEqualTo(key)) {
                if (element.getSucc() == null) {
                    element.getPred().setSucc(null);
                } else {
                    SkipListElement pred = element.getPred();
                    SkipListElement succ = element.getSucc();
                    pred.setSucc(succ);
                }
            }
        }
        return (V) tmp.getValue();
    }

    @Override
    public V getPred(K key) throws NoElementInRangeException {
        Stack<SkipListElement> stack = search(key);
        SkipListElement element = stack.pop();
        if (element.isDummy()) {
            throw new NoElementInRangeException();
        }
        if (element.isEqualTo(key)) {
            element = element.getPred();
            if (element.isDummy()) {
                throw new NoElementInRangeException();
            }
        }
        return (V) element.getValue();
    }

    @Override
    public V getSucc(K key) throws NoElementInRangeException {
        Stack<SkipListElement> stack = search(key);
        SkipListElement element = stack.pop();
        if (element.getSucc() == null) {
            throw new NoElementInRangeException();
        }
        return (V) element.getSucc().getValue();
    }

    @Override
    public int getSize() {
        int size = 0;
        SkipListElement element = firstLowest;
        while (element.getSucc() != null) {
```

```java
            size++;
            element = element.getSucc();
        }
        return size;
    }

    @Override
    public boolean isEmpty() {
        return getSize() == 0;
    }

    public int getLevel() {
        int level = 1;
        SkipListElement element = firstHighest;
        while (element.getDown() != null) {
            level++;
            element = element.getDown();
        }
        return level;
    }

    public SkipListIterator<K, V> iterator() {
        if (getSize() > 0) {
            return new SkipListIterator<>(firstLowest.getSucc());
        }
        throw new EmptyException();
    }

    public SkipListReverseIterator<K, V> reverseIterator() {
        if (getSize() > 0) {
            return new SkipListReverseIterator<>(lastLowest.getPred());
        }
        throw new EmptyException();
    }

    @Override
    public String toString() {
        StringBuilder sB = new StringBuilder();
        SkipListElement begin = firstHighest;
        int ebene = getLevel() - 1;
        do {
            sB.append(ebene + ": ");
            SkipListElement skipListElement = begin;
            while (skipListElement != null) {
                sB.append(skipListElement.toString() + ";");
                skipListElement = skipListElement.getSucc();
            }
            begin = begin.getDown();
            sB.append("\n");
            ebene--;
        } while (begin != null);
        return sB.toString();
    }
}
```

Listing 8: SkipList

(b) *Read the specification of* `java.util.Iterator`*. Write methods* `iterator()` *and* `reverseIterator()` *that create an* `Iterator`*-Object.* `iterator()` *should process the entries in sorted order,* `reverseIterator()` *should process the entries in reverse order. Make sure your implementation follows the specification laid out in the Java-API-documentation. You do not need to implement the* `remove()`*-method.*

Bei unsere Implementierung funktioniert reverse Iterator nicht, da wir keinen funktionierenden Dummy am Ende haben.

```java
package de.smuu.add.e07;

import java.util.Iterator;
import java.util.NoSuchElementException;

public class SkipListIterator<K extends Comparable<K>, V> implements
    Iterator {

    private SkipListElement<K, V> current;

    public SkipListIterator(SkipListElement node){
        current = node;
    }
    @Override
    public boolean hasNext() {
        return (current.getSucc() != null);
    }

    @Override
    public SkipListElement<K, V> next() {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }
        current = current.getSucc();
        return current;
    }
}
```

Listing 9: SkipListIterator

```java
package de.smuu.add.e07;

import java.util.Iterator;
import java.util.NoSuchElementException;

public class SkipListReverseIterator<K extends Comparable<K>, V>
    implements Iterator {

    private SkipListElement<K, V> current;

    public SkipListReverseIterator(SkipListElement node){
        current = node;
    }
    @Override
    public boolean hasNext() {
        return (current.getPred() != null);
    }

```

```
    @Override
    public SkipListElement<K, V> next() {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }
        current = current.getPred();
        return current;
    }
}
```

Listing 10: SkipListReverseIterator

(c) voluntary, 5 additional points *What happens if the skip list is changed while an* `Iterator`*-object is active? Describe a possible problem. One solution uses* fail-fast*iterators: as soon as the skip list is changed through* `put` *or* `remove`*,* all *active* `Iterator`*-objects become invalid. Later method calls result in a* `ConcurrentModificationException`*. Iterators that were created after the modification should still work. How can this behavior be implemented? Read about the* observer pattern*, and describe how to use it for the problem at hand. You do not need to write any code.*

Ein Iterator beinhaltet einen bestimmten Zustand der Datenstruktur. Wenn nun ein Element entfert wird und das das nächste Element im Interator wäre, haben wir ein Problem.

Um den Zeitpunkt des erstellen können wir die `currentTimeMillis` abspeichern und bei einem Zugriff testen, ob in der Zeit die Datenstruktur verändert wurde und dann die Exception `ConcurrentModificationException` werfen

4 Cryptographic Hash Functions (__/voluntary, 10)

(a) *What is a crytopgraphic hash function? Name three heuristic candidates.*

Eine kryptographische Hashfunktion ist eine zufällige, kollisionsresistente Hashfunktion. Theoretisch sind Kollisionen möglich, aber mit einer solchen geringen Wahrscheinlichkeit, dass es praktisch nie eintrifft.
Die Zielmenge ist in der Größenordnung so, dass man viele Kombinationsmöglichkeiten hat, ohne allzu viel Speicher zu verwenden. Dadruch kann der gehashte Wert als Fingerabdruck oder als Prüfsumme einer Eingabe verwendet werden.
Ein Gegner, der eine Hashwert knacken möchte, muss im Erwartungswert so viele Ürbilder testen, wie es Möglichkeiten zur Darstellung des Hashwertes gibt.

Beispiele hierfür sind Tiger, PANAMA und SMASH.

(b) *Which cryptographic hash functions are available in Java? How can they be used?*

In jeder Java Implemation sind drei kryptographische Hashfunktion vertreten. Darüber hinaus wurden über die verschiedene Versionen weitere Hashfunktionen hinzugefügt. MD2, MD2, SHA-1, SHA-256, SHA-512 ist eine Auswahl von oft verwendeten kryptographische Hashfunktionen.

Es gibt auch eine Möglichkeit eigene Implemationen von einer kryptographische Hashfunktion zu verwenden über die vorhandene Klasse.

Um eine kryptographische Hashfunktion in Java zu verwenden erstellt man eine neue Instanz der MessageDigest mit `h = MessageDigest.getInstance("<Hashfunktion>");`. Nun kann man mit `h.update("<text>")` einen String einfügen, der von dieser Instanz den Hashwert bildet. Mit `h.digest()` bekommt man diesen Hashwert zurückgegeben und kann dann weiter verwendet werden.

(c) *Adapt your hash table from Problem 1(b), so that it uses cryptographic hash functions. Perform appropriate experiments. Give an advantage and a disadvantage of this implementation.*

Siehe Aufgabe 1(b)